

AI Intelligence Data Ingestion & Enrichment Pipeline

System Architecture Blueprint (Designed for 500,000+ Record Horizontal Scale)

1. Executive Summary & Core System Architecture

The AI Intelligence Data Ingestion & Enrichment Pipeline is a high-throughput, fault-tolerant asynchronous system designed to collect, validate, enrich, and deduplicate AI ecosystem intelligence. The system strictly operates on a zero-hallucination policy—all data points trace directly to verified source URLs (arXiv, Y Combinator, Product Hunt, Job Boards, and AI News feeds). Unverified fields default to null/empty values.

Component	Technology Stack	Role / Functionality
Async Crawling	Python asyncio + aiohttp	Non-blocking network IO with bounded semaphore concurrency.
Browser Automation	Playwright Async	Headless Chromium fallback for JS-heavy anti-bot targets.
LLM Failover Chain	Gemini Flash → Groq → DeepSeek	Multi-tier LLM fallback for structured JSON extraction.
Data Storage	SQLite / PostgreSQL + CSV	Local checkpointing and 6-tab Google Sheets exporting.

2. Scaling Architecture for 500,000+ Records

- **Distributed Queue Model:** Decouples URL scraping from LLM extraction using Kafka / Redis queues.
- **Streaming Batch Processing:** Records are processed in bounded memory chunks (e.g. 500 items/batch) to prevent OOM errors.
- **Distributed Lock Manager:** Redis Redlock ensures multiple crawler workers never process the same URL.
- **Database Partitioning:** PostgreSQL declarative table partitioning by record_type and monthly range.

3. 413 Payload & 429 Rate Limit Resilience

- **HTML Cleaning & Chunking (413):** BeautifulSoup strips <script>, <style>, <nav>, <header>, <footer>. Documents >4,000 chars are semantically chunked before calling LLMs.
- **Exponential Backoff & Jitter (429):** Backoff delay calculated as $\text{factor}^{\text{attempt}} + \text{jitter}$, dynamically honoring HTTP Retry-After headers.
- **Tiered Provider Failover:** If Gemini (Tier 1) hits rate limits or fails, execution automatically transfers to Groq Llama (Tier 2), then DeepSeek (Tier 3).

4. Freshness & Distributed Deduplication Strategy

- **24-Hour Publication Boundary:** All job postings and news articles pass through UTC datetime normalization. Records with publication timestamps >24 hours old are strictly filtered out.
- **Canonical Fingerprinting:** Unique SHA-256 fingerprint generated from normalized URL, canonical entity name, and publication date:

$$Fingerprint = SHA256(NormalizedURL \parallel CanonicalName \parallel PubDate)$$

- **Distributed Bloom Filter:** A global Redis Bloom filter enables O(1) duplicate URL checks across N concurrent crawler nodes.

5. Primary Database & Vector/Graph Storage Blueprint

- **Primary Database (PostgreSQL):** Relational storage with JSONB payload indexing and GIN indexes for fast attribute searching.
- **Vector Search (PgVector / Qdrant):** Dense vector embeddings (text-embedding-004) stored for paper abstracts and news articles to power semantic search and topic clustering.
- **Graph Storage (Neo4j):** Graph relations modeled across entities:

(STARTUP) -[:LAUNCHED]-> (PRODUCT)
(STARTUP) -[:POSTED]-> (JOB)
(RESEARCH_PAPER) -[:HAS_REPO]-> (GITHUB_REPO)

6. Deterministic Entity Resolution Engine

Entity resolution maps raw company strings (e.g. 'Open AI, Inc.', 'Open AI') to canonical entities ('OpenAI') using a multi-stage deterministic pipeline:

1. Lowercasing and strip punctuation & whitespace.
2. Strip common corporate legal designators (Inc, LLC, Corp, Ltd, GmbH, Co, PBC).
3. Exact and tight-whitespace match against a 50-item seed AI startup database.
4. Audit log written to `6_entity_mapping_log.csv`.

7. Output & Export Pipeline (6 Google Sheets Tabs)

Tab Number	Tab Name	Schema Summary
Tab 1	Startups	Entity Name, Employee Count, Source Name, Source URL, Collected At
Tab 2	Products	Startup Name, Pricing Model (FREE FREEMIUM PAID ENTERPRISE), Source URL
Tab 3	Research Papers	Title, Authors, Paper URL, GitHub URL, GitHub Stars, Published Date
Tab 4	Jobs	Company, Date, Is Remote, Role Family, Source Name, Source URL
Tab 5	News	Title, Source Name, URL, Publication Date, Text Snippet
Tab 6	Entity Mapping Log	Raw Name, Canonical Name, Match Method

8. Data Verification & Quality Control Summary

The pipeline enforces strict automated quality gates prior to export:

Check Category	Enforcement Mechanism	Action on Violation
Source Traceability	Mandatory source.url check on every Pydantic record	Reject record immediately.
No Hallucinations	GitHub stars fetched live via GitHub REST API; no LLM estimation	Set github_stars = null.
Freshness Filter	UTC cutoff check: (now - pub_date) <= 24 hours	Discard stale job/news record.
Schema Integrity	Pydantic v2 strict type and enum validation	Log validation error & failover.

9. Production Deployment Blueprint

- **Containerization:** Packaged into a lightweight Docker container with Python 3.11-slim and headless Chromium.
- **Orchestration:** Deployed as Kubernetes CronJobs / Celery Distributed Workers for scheduled ingestion runs.
- **Observability:** Structured JSON logging with Datadog / Grafana Loki integration tracking fetch durations, HTTP status codes, LLM failovers, and entity resolution rates.